

AGBOFA

The governed digital product operating system. Intelligence is not authority.

Document issue v5.0

Received freeze -- typeset as submitted

Day 0 -- pre-implementation freeze

Human Owner + Senior System Architect

AGBOFA is not a Claude wrapper. Claude is replaceable intelligence. AGBOFA owns authority, governance, execution, verification, evidence, audit, memory, and recovery.

00 AGBOFA Master Architecture & Constitution

The governed digital product operating system.

Document issue: v5.0 -- Received freeze -- as submitted

Status: Frozen -- Day 0 -- pre-implementation

Classification: Constitution -- Supreme specification

AGBOFA is not a Claude wrapper. Claude is replaceable intelligence. AGBOFA owns authority, governance, execution, verification, evidence, audit, memory, and recovery.

You are reading the received freeze. This edition is document issue v5.0, typeset without rewriting. The operative engineering issue is v5.1 -- same constitution, clarified numbering and Day-60 scope. Switch issues in the header. Do not call them "Version 1" and "Version 2": those numbers already belong to the historical lineage v1.0--v4.0.

Part | Owns

- A Constitution | What must never be violated
- B Reference architecture | What AGBOFA is
- C Operating model | How AGBOFA works
- D Technical specification | How it is implemented
- E Enterprise target | What AGBOFA becomes at maturity
- F Implementation roadmap | How we get there
- G Change control | How the architecture itself evolves

01 Versioning law

Five numbering systems. They are not interchangeable.

This freeze is document issue v5.0. Historical issues v1.0 through v4.0 are superseded lineage. Semver in this freeze: major for constitutional change, minor for new capability, patch for fixes.

Ambiguity left in v5.0. v5.0 uses "L0--L5" for agent autonomy (Part A.4) and again for enterprise maturity (Part E.1). Those are different scales. v5.1 splits them: autonomy A0--A5, maturity M0--M5. Until you switch to v5.1, read every "L" against its section heading.

Written in v5.0 | Where | Means

- v5.0 | Cover, history | This constitutional freeze
- L0--L5 (Part A) | Autonomy levels | What an agent may do
- L0--L5 (Part E) | Enterprise maturity | How much of the target architecture is operational
- E0--E4 | Evidence | Strength of proof
- Phase / Block / Gate / Day | Part F | First increment schedule

02 How to read this document

Architecture is permanent. The roadmap is an increment.

The received freeze already divides the work into Parts A--G and places the 60-day program after the architecture. Read the Constitution first. Treat Part F as the initial build sequence. Treat Part E as the destination.

Known risk in the received freeze. Part F.1 currently says that by Day 60 every enterprise function must have an implemented counterpart, with a target of 100% architectural implementation. That sentence can make an implementer treat multi-region, marketplace, federation, and hundreds of agents as Day-60 work -- or, conversely, as "not in the architecture." v5.1 corrects the sentence without shrinking the vision.

03 What v5.1 changes -- and what it does not

Minor issue of the received freeze. Articles unchanged.

You are on v5.0. No editorial change is applied in this edition. Switch to v5.1 for the issue log.

A.1 Preamble

AGBOFA is a governed digital product operating system. This Constitution is supreme.

AGBOFA (Autonomous Governed Blockchain-Oriented Factory Architecture) is established as a governed digital product operating system. This Constitution is the supreme authority governing all actors, systems, agents, models, and processes within the AGBOFA ecosystem.

The purpose of this Constitution is to ensure that artificial intelligence may autonomously design, build, and operate digital products safely, verifiably, and constitutionally, while human sovereignty remains absolute and every consequential action carries evidence, accountability, and the possibility of audit.

Supremacy. No agent, model, subsystem, or process may violate this Constitution. No authority may be exercised except through the mechanisms defined herein.

A.2 Permanent architectural principles

P1--P15 do not change with maturity, timeline, or provider.

These principles are permanent. They do not change with maturity level, implementation timeline, or provider selection.

ID | Principle | Statement

- P1 | Human Sovereignty | Humans are ultimately accountable for AGBOFA's existence and major consequential actions.
- P2 | Constitutional Supremacy | The Constitution governs all actors, including AGBOFA itself. No agent, model, or process may violate it.
- P3 | Intelligence Is Not Authority | No AI provider, model, agent, or generated output automatically possesses authority. Authority must be explicitly granted through constitutional mechanisms.
- P4 | Authority Is Explicit and Traceable | Every consequential action requires an identifiable authority source that can be traced through the governance chain.
- P5 | Capability Is Scoped and Constrained | An actor can only perform actions for which it has been granted both authority and technical capability, within defined boundaries.
- P6 | Execution Is Isolated | Intelligence never receives unrestricted execution access. All execution occurs through governed, isolated channels.
- P7 | Verification Is Independent | Important outcomes must be independently verified before they are accepted as true or successful.
- P8 | Evidence Precedes Trust | Claims about system state, product quality, or operational success require evidence. Trust is earned, not assumed.
- P9 | Audit Is Persistent and Tamper-Evident | Consequential actions create durable audit records that cannot be silently modified or deleted.
- P10 | Governance Cannot Be Weakened by the Governed | No agent, model, or subsystem may modify the rules that

govern its own authority.

P11 | Autonomy Is Earned | Autonomy increases only according to demonstrated evidence of reliability, safety, and constitutional compliance.

P12 | Learning Requires Verified Outcomes | AGBOFA cannot convert an AI assumption or recommendation into institutional knowledge merely because an AI generated it.

P13 | Tenant Isolation Is Absolute | Products, tenants, organizations, and environments are isolated by default. Cross-boundary access requires explicit, governed authorization.

P14 | Recovery Is Constitutional | The system must be capable of freezing, recovering, rolling back, and restoring critical state. Recovery procedures are themselves governed.

P15 | Provider Independence | AGBOFA's architecture never equates its intelligence with any specific AI provider. Providers are replaceable through defined contracts.

A.3 Constitutional articles

Articles 1--16. The operative law of every actor, agent, and subsystem.

The sixteen articles are the operative law of AGBOFA. They bind every actor. They do not change with maturity, timeline, or provider.

Article 1 Human Sovereignty

Humans remain ultimately accountable for AGBOFA's existence, operation, and major consequential actions.

Scope. All AGBOFA operations, decisions, and actions.

Prohibitions

- AGBOFA shall not take actions that remove or diminish human accountability.
- No agent may claim to act without human oversight for constitutionally significant actions.

Requirements

- Human approval is required for all actions classified as HIGH or CRITICAL risk.
- A designated human owner must possess emergency freeze authority at all times.
- Human decisions that override agent recommendations must be recorded with rationale.

Enforcement. The Trust Kernel shall reject any action classified as HIGH or CRITICAL risk that lacks valid human approval.

Violation consequence. System freeze and constitutional review.

Article 2 AGBOFA Sovereignty

AGBOFA controls the ecosystem's authority and governance mechanisms.

Scope. All authority grants, governance rules, policy definitions, and capability assignments.

Prohibitions

- No external system may directly grant authority within AGBOFA.
- No agent may self-grant authority.

Requirements

- All authority flows through the AGBOFA Governance Engine.
- All governance rules are versioned and stored in the Constitution repository.
- All authority grants are recorded as audit events.

Enforcement. The Authority Kernel shall reject any authority grant that does not originate from an authorized governance source.

Violation consequence. Invalid authority revocation and security investigation.

Article 3 Intelligence Is Not Authority

No AI provider, model, agent, recommendation, inference, or generated output automatically receives authority.

Scope. All AI-generated outputs, recommendations, plans, and proposals.

Prohibitions

- AI outputs shall not be treated as commands.
- AI recommendations shall not be auto-executed without governance evaluation.
- AI confidence shall not substitute for authority.

Requirements

- All AI outputs enter the system as PROPOSALS.
- Proposals must pass through governance evaluation before execution.
- The distinction between intelligence and authority must be explicit in every agent interaction.

Enforcement. The Execution Broker shall reject any action whose authority chain does not trace to a valid, non-AI authority source.

Violation consequence. Action rejection and agent capability suspension.

Article 4 Authority Is Explicit

Every consequential action requires an identifiable authority source.

Scope. All consequential actions within AGBOFA.

Prohibitions

- Implicit authority is prohibited.
- "The agent knew what to do" is not an authority source.
- Default allow is prohibited.

Requirements

- Every action carries an authority chain.
- Authority chains must be traceable to a human or constitutional source.
- Authority grants must have defined scope, duration, and revocation conditions.

Enforcement. Actions lacking valid authority chains are rejected at the Trust Kernel.

Violation consequence. Action rejection and audit flag.

Article 5 Capability Is Scoped

An actor can only perform actions for which it possesses the required capability.

Scope. All actor actions within AGBOFA.

Prohibitions

- Broad capabilities (e.g., "agent has access to everything") are prohibited.
- Capability escalation without governance approval is prohibited.
- Capability reuse across product boundaries is prohibited by default.

Requirements

- All capabilities are defined with explicit scope, environment, and expiry.
- Capability tokens are required for all execution actions.
- Capability tokens are short-lived and renewable only through governance.

Enforcement. The Capability Manager shall reject any execution attempt with missing, expired, or out-of-scope capability tokens.

Violation consequence. Capability revocation and security audit.

Article 6 Execution Is Isolated

Intelligence does not receive unrestricted execution access.

Scope. All execution activities.

Prohibitions

- Direct agent access to production systems is prohibited.
- Execution outside isolated workspaces is prohibited.
- Bypassing the Execution Broker is prohibited.

Requirements

- All execution occurs through the Execution Broker.
- All execution occurs in isolated workspaces.
- All execution carries resource limits, timeouts, and failure handling.

Enforcement. The Execution Broker shall reject any direct execution attempt that bypasses its governance chain.

Violation consequence. Execution rejection and agent quarantine.

Article 7 Execution Is Not Success

An executed action does not constitute proof of correctness.

Scope. All execution outcomes.

Prohibitions

- Treating "the command ran" as "the command was correct."
- Accepting agent claims of success without independent verification.
- Skipping verification for consequential actions.

Requirements

- All consequential actions require independent verification.
- Verification must be performed by a component separate from the executing agent.
- Verification results must be recorded as evidence.

Enforcement. The Verification Engine shall flag any consequential action lacking verification evidence.

Violation consequence. Action marked unverified and blocked from promotion.

Article 8 Verification Is Independent

Important outcomes must be independently verified.

Scope. All important outcomes, including builds, tests, deployments, and data operations.

Prohibitions

- Agents verifying their own work.
- Verification by the same component that performed the action.
- Verification that only checks surface-level success.

Requirements

- Verification is performed by dedicated verification agents or systems.
- Verification checks both functional correctness and constitutional compliance.
- Verification produces evidence that is stored and auditable.

Enforcement. Outcomes lacking independent verification are blocked from advancing to the next lifecycle stage.

Violation consequence. Outcome rejection and re-execution requirement.

Article 9 Evidence Precedes Trust

Claims about system state require evidence.

Scope. All claims about system state, product quality, or operational success.

Prohibitions

- Accepting agent assertions without evidence.
- Treating absence of errors as evidence of correctness.
- Storing claims without supporting evidence.

Requirements

- Every consequential claim carries evidence.
- Evidence must be machine-verifiable where possible.
- Evidence must be stored in the Evidence Store with proper provenance.

Enforcement. Claims lacking evidence are marked unverified and cannot be used for decision-making.

Violation consequence. Claim rejection and investigation.

Article 10 Audit Is Persistent

Consequential actions create durable audit records.

Scope. All consequential actions.

Prohibitions

- Audit record modification or deletion.
- Audit bypass for any reason.
- Tampering with audit timestamps, hashes, or chains.

Requirements

- All consequential actions generate audit events.
- Audit events are append-only.
- Audit events are hash-chained for tamper evidence.
- Audit records include: who, which agent, which authority, which capability, which product, which environment, what action, when, why, what happened, what was verified, what evidence exists.

Enforcement. The Audit System shall detect any tampering attempt and trigger a security alert.

Violation consequence. System freeze and security investigation.

Article 11 Governance Cannot Be Weakened by the Governed

An agent cannot modify the rules that govern its own authority.

Scope. All governance rules, policies, and authority definitions.

Prohibitions

- Agent modification of its own authority scope.
- Agent modification of governance rules that apply to itself.
- Agent creation of new capabilities that bypass governance.

Requirements

- Governance changes require human approval.
- Governance changes are versioned and audited.
- Governance changes are subject to constitutional review.

Enforcement. The Governance Engine shall reject any governance change originating from a governed agent.

Violation consequence. Change rejection and agent suspension.

Article 12 Autonomy Is Earned

Autonomy increases only according to defined evidence and policy.

Scope. All agent autonomy levels.

Prohibitions

- Automatic autonomy grants.
- Autonomy increases without evidence of reliability.
- Autonomy based on AI self-assessment.

Requirements

- Autonomy levels are defined in the Constitution.
- Autonomy increases require evidence of successful governed operation.
- Autonomy increases require human approval.

Enforcement. The Trust Kernel shall enforce autonomy levels and reject actions exceeding authorized autonomy.

Violation consequence. Action rejection and autonomy level reduction.

Article 13 Learning Requires Verified Outcomes

AGBOFA cannot convert an AI assumption into institutional knowledge merely because the AI generated it.

Scope. All institutional knowledge and learning.

Prohibitions

- Direct promotion of AI outputs to knowledge without verification.
- Treating AI confidence as knowledge validity.
- Storing unverified claims in institutional memory.

Requirements

- Knowledge candidates must pass through verification.
- Verified outcomes must be linked to their evidence.
- Knowledge must be versioned and auditable.

Enforcement. The Memory System shall reject knowledge candidates lacking verification evidence.

Violation consequence. Knowledge candidate rejection.

Article 14 Tenant Isolation

One product, tenant, organization, or environment cannot improperly access another.

Scope. All cross-tenant, cross-product, and cross-environment interactions.

Prohibitions

- Cross-tenant data access by default.
- Cross-product agent access by default.
- Cross-environment capability use by default.

Requirements

- All tenant data is isolated by default.
- Cross-boundary access requires explicit, governed authorization.
- Tenant isolation is tested continuously.

Enforcement. The Isolation Guard shall detect and block any cross-tenant access attempt lacking explicit authorization.

Violation consequence. Access denial, isolation breach alert, and security investigation.

Article 15 Recovery Is Constitutional

The system must be capable of freezing, recovering, rolling back, and restoring critical state.

Scope. All critical state, including products, databases, configurations, and governance rules.

Prohibitions

- Operating without defined recovery procedures.
- Recovery procedures that bypass governance.
- Recovery actions that are not audited.

Requirements

- Recovery procedures are defined for all critical state.
- Recovery actions are authorized and audited.
- Recovery testing occurs regularly.
- Emergency freeze capability exists and is testable.

Enforcement. The Recovery System shall reject recovery actions lacking proper authorization and audit trail.

Violation consequence. Recovery action rejection and constitutional review.

Article 16 Provider Independence

AGBOFA's architecture never equates its intelligence with any specific AI provider.

Scope. All intelligence integration points.

Prohibitions

- Hard-coding provider-specific APIs in core AGBOFA components.
- Requiring provider-specific features for constitutional operation.
- Treating any provider as irreplaceable.

Requirements

- All intelligence access flows through the Intelligence Contract.
- Provider adapters are pluggable and versioned.
- Provider switching is possible without core changes.

Enforcement. The Intelligence Contract shall reject any direct provider access that bypasses the abstraction layer.

Violation consequence. Architecture violation flag and refactoring requirement.

A.4 Autonomy levels

Autonomy is earned. No agent starts above proposal. L in the freeze is autonomy, not maturity.

The received freeze (v5.0) published a single L0--L5 ladder here. That ladder is agent autonomy, not enterprise maturity. v5.1 splits the numbering: autonomy uses A0--A5; enterprise maturity uses M0--M5 (Part E). Evidence stays E0--E4.

L is autonomy, not maturity. Keep this table as the freeze wrote it. Do not read L3 here as enterprise multi-product maturity. The same letter was reused in Part E for a different scale. The operative issue replaces L with A for this table.

Level | Name | Description | Requirements

- Level: L0
- Name: Advisory
- Description: Agent provides recommendations only. No execution authority.
- Requirements: None -- default state

- Level: L1
- Name: Proposal
- Description: Agent can submit formal proposals for governance evaluation.
- Requirements: Proven recommendation quality
- Level: L2
- Name: Governed Execution
- Description: Agent can execute within pre-approved scope with governance checks.
- Requirements: Evidence of safe L1 operation
- Level: L3
- Name: Autonomous Within Scope
- Description: Agent can execute without per-action approval within defined scope.
- Requirements: Evidence of safe L2 operation + human approval
- Level: L4
- Name: Autonomous with Verification
- Description: Agent can execute and self-verify within scope, with periodic external verification.
- Requirements: Evidence of safe L3 operation + human approval
- Level: L5
- Name: Full Governed Autonomy
- Description: Agent can operate autonomously within constitutional limits.
- Requirements: Evidence of safe L4 operation + human approval + constitutional review

Default and increase. No agent starts above L1. Autonomy increases require evidence and human approval. Autonomy based on AI self-assessment is prohibited (Article 12).

A.5 Risk classifications

CRITICAL, HIGH, MEDIUM, LOW. Approval is a function of class, not of model confidence.

Every consequential action is classified before governance evaluation. Classification is a Trust Kernel function. AI confidence is not a risk class.

Classification | Description | Approval required

CRITICAL | Actions affecting constitutional integrity, production data, security boundaries, or system-wide state. | Human + Constitutional Review

HIGH | Actions affecting product deployments, schema changes, credential operations, or cross-tenant access. | Human

MEDIUM | Actions affecting code changes, test execution, or non-production environments. | Governance Engine

LOW | Actions affecting documentation, analysis, or read-only operations. | Capability Token only

Default-deny. Unclassified consequential actions are treated as HIGH until classified. Default-allow is prohibited (Article 4).

B.1 System overview

Human -> Constitution -> Governance -> Authority -> Capabilities -> Intelligence -> Planning -> Execution -> Verification -> Evidence -> Audit -> Memory -> Learning.

Authority flows downward. Evidence and audit flow back up. No layer may skip the layers above it. Intelligence sits below capability; it never sits above authority.

1. Human -- Sovereign Authority
2. Constitution -- Supreme Governing Law
3. Governance -- Policy & Rule Engine

4. Authority -- Explicit Grant System
5. Capabilities -- Scoped Token System
6. Intelligence -- AI Provider Abstraction
7. Planning -- Governed Action Plans
8. Execution -- Isolated Broker / Workspace
9. Verification -- Independent Validation
10. Evidence -- Immutable Proof Store
11. Audit -- Append-Only Trail
12. Memory -- Verified Knowledge
13. Learning -- Institutional Growth

Read direction. A proposal originates in Intelligence or Planning and must climb back through Governance and Authority before Execution will admit it. Learning cannot grant autonomy or rewrite the Constitution.

B.2 Core subsystems

The kernels and engines that enforce the Constitution in software.

Subsystems are the implementable counterparts of the stack. Each maps to one or more articles. Absence of a subsystem is a constitutional defect, not a backlog item.

Subsystem | Purpose | Key components

Trust Kernel | Enforce constitutional rules | Auth, Authorization, Policy Engine, Risk Classification

Authority System | Manage explicit authority grants | Authority Chains, Delegation, Revocation

Capability System | Issue scoped capability tokens | Token Issuance, Scope Validation, Expiry Management

Execution Broker | Control all execution | Action Classification, Governance Check, Workspace Allocation

Verification Engine | Independently verify outcomes | Test Runners, Build Verification, Deployment Verification

Evidence Store | Store immutable evidence | Hash-chained Storage, Evidence Validation

Audit System | Maintain append-only audit trail | Event Logging, Hash Chaining, Tamper Detection

Intelligence Contract | Abstract AI providers | Provider Adapters, Agent Runtime, Context Management

Product Factory | Transform ideas into products | Discovery, Requirements, Blueprint, Architecture, Planning

Memory System | Store verified knowledge | Knowledge Candidates, Validation, Institutional Memory

Recovery System | Enable constitutional recovery | Freeze, Rollback, Restore, Emergency Procedures

Operations System | Manage running products | Monitoring, Incident, Compliance, Lifecycle

B.3 Intelligence architecture

Providers are adapters. Outputs are proposals. Autonomy is A, not L.

All model access flows through the Intelligence Contract. Provider adapters are pluggable. Agent outputs enter the system as proposals, never as commands (Article 3, Article 16).

1. Intelligence provider
2. Provider adapter
3. Intelligence Contract
4. Agent runtime
5. Agent role
6. Agent context
7. Agent task
8. Recommendation / plan / output (PROPOSAL)

Initial agent roles

The received freeze numbered these roles L1 / L2. That L is the autonomy ladder from A.4, not enterprise maturity from E.1. v5.1 reprints the same roster on the A scale.

Agent | Purpose | Autonomy

Product Discovery Agent | Transform ideas into requirements | L1

Requirements Agent | Define and validate requirements | L1

Domain Analyst Agent | Analyze domain models | L1

Architecture Agent | Propose system architecture | L1

Database Agent | Design and manage data models | L1

UX Agent | Design user experience | L1

Security Agent | Identify and address security concerns | L1

Repository Agent | Manage code repositories | L1

QA Agent | Design and execute tests | L1

Verification Agent | Independently verify outcomes | L2

Deployment Agent | Manage deployments | L1

Operations Agent | Monitor and manage operations | L1

B.4 Product factory lifecycle

Idea to retirement under governance. v5.1 adds Discovery, Requirements, and Architecture state machines.

The factory is a governed pipeline, not a prompt. Human approval sits after architecture and before implementation. Verification and evidence sit after execution and before deployment.

1. Idea
2. Discovery
3. Requirements
4. Domain model
5. Product Passport
6. Blueprint
7. Architecture
8. Human approval
9. Implementation plan
10. Execution
11. Build
12. Test
13. Security
14. Verification
15. Evidence
16. Deployment approval
17. Deployment
18. Post-deployment verification
19. Operations
20. Learning
21. Improvement
22. Retirement

B.5 Execution path

Proposal to audit. Nothing runs that has not passed classification, governance, authority, capability, and approval.

Execution is admitted only through the Execution Broker, only inside an isolated workspace, and only against a valid authority chain and in-scope capability token (Articles 4, 5, 6). A result is a claim, not success (Article 7).

1. Agent proposal
2. Action classification
3. Risk assessment
4. Governance evaluation
5. Authority check
6. Capability check
7. Approval check
8. Capability token
9. Execution Broker
10. Isolated workspace
11. Worker
12. Result
13. Verification
14. Evidence
15. Audit

B.6 Verification and evidence

Independent checks. Evidence levels E0--E4. E0 is unacceptable for every consequential claim.

Verification types

Type | Purpose | Performed by

Functional | Verify functional requirements | Verification Agent
Unit | Verify individual components | Automated Test Runner
Integration | Verify component interactions | Automated Test Runner
E2E | Verify user workflows | Playwright Runner
Security | Verify security controls | Security Agent
Performance | Verify performance requirements | Performance Runner
Deployment | Verify deployment success | Deployment Verifier
Data Integrity | Verify data operations | Data Verifier
Recovery | Verify recovery procedures | Recovery Tester
Configuration | Verify configuration correctness | Configuration Verifier
Compliance | Verify constitutional compliance | Compliance Checker

Evidence levels

Evidence numbering is E0--E4 in both the received freeze and the operative issue. It is not renamed. It is not autonomy and not maturity.

Level | Description | Required for

E0 | No evidence -- agent assertion only | Nothing (unacceptable)

E1 | Automated test output | LOW risk actions

E2 | Test output + configuration snapshot | MEDIUM risk actions

E3 | Test output + configuration + security scan | HIGH risk actions

E4 | Full evidence package + independent verification | CRITICAL risk actions

E0 is never enough. Claims lacking evidence are marked unverified and cannot be used for decision-making (Article 9). Absence of errors is not evidence of correctness.

B.7 Memory and learning

AI saying something does not become AGBOFA knowledge.

1. Raw observation
2. Evidence
3. Verified outcome
4. Knowledge candidate
5. Validation
6. Institutional knowledge
7. Future decision support

Verified outcomes only. AI saying something does not become AGBOFA knowledge. Only verified outcomes with evidence become knowledge (Article 13). Learning never grants authority and never weakens governance.

C.1 Daily operating rhythm

Eight hours. Brief, implement, assemble, test, attack, freeze.

The daily rhythm is how humans and agents share a clock without sharing authority. Architecture decides what should exist before agents generate. Evidence is frozen before the day closes.

Time | Activity | Hours | Purpose

- Time: 08:00--09:00
- Activity: Architecture & Command Briefing
- Hours: 1.0
- Purpose: Decide what should exist
- Time: 09:00--11:30
- Activity: AI Agent Implementation
- Hours: 2.5
- Purpose: Agents generate and modify systems
- Time: 11:30--13:00
- Activity: Assembly & Integration
- Hours: 1.5
- Purpose: Connect components
- Time: 14:00--15:30
- Activity: Testing & Validation
- Hours: 1.5
- Purpose: Prove functionality
- Time: 15:30--16:15
- Activity: Security & Governance Verification
- Hours: 0.75
- Purpose: Attack and verify controls

- Time: 16:15--17:00
- Activity: Evidence & Daily Freeze
- Hours: 0.75
- Purpose: Preserve decisions and evidence
- Time: TOTAL
- Activity:
- Hours: 8.0
- Purpose:

C.2 Weekly rhythm and six-day cycle

Heavy assembly, then integration, then attack. Every block: plan, assemble, test, attack, verify, freeze.

Weekly rhythm

Days | Mode

Monday--Thursday | Heavy assembly

Friday | Integration + testing

Saturday | Security + verification + documentation

Sunday | Light review / planning / recovery (2--4 hours)

Six-day work cycle

Each implementation block follows the same six beats. Skipping ATTACK or FREEZE is a process violation, not a schedule compression.

1. Plan
2. Assemble
3. Test
4. Attack
5. Verify
6. Freeze

C.3 Ten gates

Gate 1 constitutional runtime through Gate 10 red-team certification. Gates are evidence, not dates.

A gate is a certification event with an evidence package. Passing a calendar day does not pass a gate. Gate numbers are delivery checkpoints; they are not autonomy grants, not maturity levels, and not document-issue numbers.

Gate | Description | Blocks covered

Gate 1 | Constitutional Runtime works | Block 1

Gate 2 | Trust Kernel works | Block 2

Gate 3 | Authority / Capability security works | Block 3

Gate 4 | AI Agent runtime works | Block 4

Gate 5 | Product specification factory works | Block 5

Gate 6 | Controlled execution works | Block 6

Gate 7 | Independent verification works | Block 7

Gate 8 | Deployment lifecycle works | Block 8

Gate 9 | Multiple products remain isolated | Block 9

Gate 10 | Red-team certification passes | Block 10

C.4 Layer contracts

Purpose, permissions, and failure modes for every layer in the stack.

The received freeze defined the stack in Part B and the rhythm in Part C. It did not publish a per-layer contract catalogue.

Catalogue lives in the operative issue. The layer-by-layer contract -- purpose, responsibilities, inputs, outputs, permissions, prohibitions, dependencies, data, failures, security, audit -- is issued in v5.1. Switch to the v5.1 operative issue to read it.

C.5 Agent catalogue

Named agents, tools, prohibitions, and autonomy grants A0 / A1 / A2.

The received freeze listed initial agent roles in B.3 with L1 / L2 autonomy cells. It did not publish per-agent contracts (inputs, outputs, tools, prohibitions, confidence, autonomy).

Catalogue lives in the operative issue. The agent-by-agent contract is issued in v5.1, using A0--A2 grants. Switch to the v5.1 operative issue to read it. L1 in the freeze is A1; L2 is A2.

D.1 Repository structure

Constitution at the root. Core, intelligence, factory, products, operations, evidence, command center.

The repository is itself a constitutional artifact. The Constitution directory is supreme. Products never import core internals that would let an agent self-grant authority.

```
agbofa/
??? constitution/           # THE CONSTITUTION (supreme)
?   ??? articles/          # Each article as machine-readable contract
?   ??? principles/        # Permanent architectural principles
?   ??? amendments/        # Constitutional amendment process
?   ??? version-control/    # Constitutional versioning
?
??? core/                   # AGBOFA CORE PLATFORM
?   ??? trust-kernel/      # Auth, governance, policy engine
?   ??? authority/         # Authority model, delegation, revocation
?   ??? capability/        # Capability tokens, scopes, expiry
?   ??? execution/         # Broker, workspace, workers, sandbox
?   ??? verification/      # Independent verification engine
?   ??? evidence/          # Evidence collection, hashing, storage
?   ??? audit/             # Append-only audit log
?   ??? recovery/          # Freeze, rollback, restore
?
??? intelligence/           # AI ABSTRACTION LAYER
?   ??? contract/          # Intelligence Contract (provider-agnostic)
?   ??? adapters/          # Provider adapters
?   ??? runtime/           # Agent runtime, context, task management
?   ??? memory/            # Institutional memory and learning
?
??? factory/                # PRODUCT FACTORY
?   ??? discovery/         # Product discovery workflow
?   ??? requirements/      # Requirements engine
?   ??? blueprint/         # Product blueprint generation
?   ??? architecture/      # Architecture proposal system
?   ??? planning/          # Implementation planning
?
??? products/               # PRODUCTS BUILT BY AGBOFA
```

```

?   ??? product-a/
?   ?   ??? passport.json      # Product Passport (identity contract)
?   ?   ??? requirements/
?   ?   ??? blueprint/
?   ?   ??? source/
?   ?   ??? evidence/
?   ?   ??? audit/
?   ??? product-b/
?
??? operations/                # ENTERPRISE OPERATIONS
?   ??? monitoring/
?   ??? incident/
?   ??? deployment/
?   ??? compliance/
?
??? templates/                # TECHNOLOGY TEMPLATES
?   ??? web-app/
?   ??? api-service/
?   ??? database/
?
??? evidence/                  # AGBOFA'S OWN EVIDENCE
?   ??? daily-freezes/
?   ??? gate-certifications/
?   ??? red-team/
?   ??? completion-matrix.json
?
??? command-center/           # AGBOFA COMMAND CENTER
  ??? api/
  ??? dashboard/
  ??? cli/

```

D.2 Technology stack

TypeScript, Node 20, Postgres 16, SQLite, Drizzle, BullMQ, XState 5, Zod, Vitest, Playwright, Fastify, JWT and capability tokens, Docker Compose (phase 1), React and Vite.

The stack is chosen for type safety, explicit state, and auditable storage. Provider-specific AI SDKs do not appear in core. Phase 1 deployment is a single-node Docker Compose so the execution path remains inspectable.

Component | Technology | Rationale

Language | TypeScript (strict mode) | Type safety, ecosystem maturity

Runtime | Node.js 20+ | Production-ready, widespread support

Database (Primary) | PostgreSQL 16 | Enterprise-grade, ACID compliance

Database (Isolated) | SQLite | Lightweight test and workspace environments

ORM | Drizzle ORM | SQL-first, TypeScript-native, lightweight

Queue | BullMQ (Redis-based) | Production-ready job queue

State management | XState 5 | Explicit state machines, TypeScript-native

Validation | Zod | Runtime validation, TypeScript inference

Testing (Unit / Integration) | Vitest | Fast, modern, TypeScript-native

Testing (E2E) | Playwright | Browser automation, cross-browser

API framework | Fastify | High-performance, TypeScript-native

Authentication | Custom JWT + capability tokens | Constitutional authority model

Audit storage | PostgreSQL + append-only tables | Durable, queryable, hash-chained

Containerization | Docker + Docker Compose | Consistent environments

Deployment (Phase 1) | Single-node Docker Compose | Controlled, auditable

CLI | Commander.js | Mature CLI framework

[Dashboard](#) | [React + Vite](#) | [Command center UI](#)

[Documentation](#) | [Markdown](#) + [OpenAPI 3.1](#) | [Machine-readable API docs](#)

D.3 Product Passport

Identity contract for every product. v5.1 binds constitutionVersion and a closed lifecycleState union.

Every product carries a Product Passport. The passport is the identity contract: tenant, owner, environments, agents, capabilities, evidence, recovery. The received freeze typed lifecycleState as an open string.

```
interface ProductPassport {
  productId: string;
  productName: string;
  tenantId: string;
  ownerId: string;
  version: string;
  lifecycleState: string;
  requirements: RequirementReference[];
  architecture: ArchitectureReference;
  technologyProfile: TechnologyProfile;
  database: DatabaseReference;
  repositories: RepositoryReference[];
  environments: EnvironmentReference[];
  deployments: DeploymentReference[];
  agents: AgentReference[];
  capabilities: CapabilityReference[];
  policies: PolicyReference[];
  auditHistory: AuditReference[];
  evidence: EvidenceReference[];
  incidents: IncidentReference[];
  recoveryState: RecoveryReference;
}
```

D.4 Capability token and audit event

Scoped, short-lived capability. Append-only, hash-chained audit.

Capability token

An actor can only perform actions for which it possesses the required capability (Article 5). Tokens are short-lived, scoped to product and environment, and renewable only through governance.

```
interface CapabilityToken {
  tokenId: string;
  capability: string;           // e.g., "repo.read"
  scope: {
    productId: string;
    environment: string;
    resource?: string;
  };
  grantedTo: {
    agentId: string;
    role: string;
  };
  grantedBy: {
    authorityChain: string[];
    humanApproval?: string;
  };
  issuedAt: string;
  expiresAt: string;
  constraints: {
    maxDuration?: number;
    maxOperations?: number;
    allowedCommands?: string[];
  };
}
```

```
};  
}
```

Audit event

Consequential actions create durable audit records (Article 10). Events are append-only and hash-chained. Modification, deletion, or bypass is a security incident.

```
interface AuditEvent {  
  eventId: string;  
  timestamp: string;  
  actor: {  
    type: "human" | "agent" | "system";  
    id: string;  
    role?: string;  
  };  
  action: {  
    type: string;  
    classification: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";  
    description: string;  
  };  
  authority: {  
    chain: string[];  
    token?: string;  
    humanApproval?: string;  
  };  
  capability: {  
    name: string;  
    scope: string;  
    tokenId?: string;  
  };  
  target: {  
    productId?: string;  
    environment?: string;  
    resource?: string;  
  };  
  result: {  
    status: "success" | "failure" | "rejected" | "error";  
    details?: string;  
  };  
  verification: {  
    performed: boolean;  
    method?: string;  
    result?: string;  
    evidenceId?: string;  
  };  
  hash: string;  
  previousHash: string;  
}
```

D.5 Intelligence Contract

Provider-agnostic contract. Adapters for Claude, GPT, and Local are replaceable.

All intelligence access flows through the Intelligence Contract (Article 16). The contract names capabilities and constraints, not a vendor. Direct provider access from core is an architecture violation.

```
interface IntelligenceContract {  
  contractId: string;  
  version: string;  
  provider: string;  
  capabilities: {  
    reasoning: boolean;  
    codeGeneration: boolean;  
    analysis: boolean;  
    planning: boolean;  
  };  
}
```

```

    toolUse: boolean;
  };
  constraints: {
    maxContextLength: number;
    maxOutputLength: number;
    timeoutMs: number;
    retryPolicy: RetryPolicy;
  };
  security: {
    dataIsolation: boolean;
    noTrainingOnData: boolean;
    auditLogging: boolean;
  };
}

```

E.1 Enterprise maturity

Platform capability scale. The freeze wrote L0--L5. The operative issue writes M0--M5. M is not A.

The received freeze published enterprise maturity as L0--L5. The same L prefix was used in A.4 for agent autonomy. That collision is a numbering defect. Read this table as platform maturity, not as an agent grant.

Numbering collision. v5.1 splits the ladder. Autonomy is A0--A5. Maturity is M0--M5. Evidence stays E0--E4. An L number from the freeze is not usable until you know which table it came from.

Level | Name | Description | Key capabilities

- Level: L0
- Name: Constitutional Foundation
- Description: Constitution defined, basic governance exists
- Key capabilities: Trust Kernel, Authority, Capability
- Level: L1
- Name: Core Platform
- Description: Platform can govern and execute
- Key capabilities: Execution Broker, Verification, Evidence
- Level: L2
- Name: Product Factory
- Description: Platform can build products
- Key capabilities: Discovery, Requirements, Blueprint, Planning
- Level: L3
- Name: Multi-Product Ecosystem
- Description: Multiple products managed simultaneously
- Key capabilities: Tenant Isolation, Portfolio Management
- Level: L4
- Name: Enterprise Platform
- Description: Enterprise-grade operations
- Key capabilities: Monitoring, Incident, Compliance, Recovery
- Level: L5
- Name: Autonomous Governed Ecosystem
- Description: Full governed autonomy
- Key capabilities: Self-improvement, Advanced Learning

E.2 Enterprise capabilities

Target-state capabilities. Architecture-ready is not Day-60 operational.

Phase 1 implements the architecture that supports these capabilities. Full operational deployment of every enterprise feature may require additional infrastructure beyond Day 60, but the architectural support is present.

Capability | Description | Implementation status

Multi-Region Infrastructure | Deploy across geographic regions | Phase 1: Architecture-ready, single-node implementation

Multi-Provider AI Routing | Route intelligence to different providers | Phase 1: Contract + Adapter pattern

Enterprise SDK | External integration SDK | Phase 1: API-first architecture

Federation | Multi-instance coordination | Phase 1: Architecture-ready

Marketplace | Product / agent marketplace | Phase 1: Architecture-ready

Hundreds of Agents | Massive agent scaling | Phase 1: Agent registry scalable

Advanced Analytics | Enterprise analytics | Phase 1: Evidence-based foundation

Compliance Frameworks | SOC2, GDPR, etc. | Phase 1: Audit-ready architecture

E.3 Security constitution

Named threats, articles, and controlling subsystems.

The received freeze distributed security across articles, red-team scenarios, and gate criteria. It did not publish a named threat table as part of the enterprise target.

Threat table lives in the operative issue. v5.1 names the threats, the articles they violate, and the subsystems that must block them. Switch to the v5.1 operative issue to read the table. Red-team IDs RT-001--RT-010 remain the test bindings in Part F.

F.1 Program objective

The freeze claimed 100% at Day 60. The operative issue retargets certification.

At the end of Day 60, the received freeze required:

Every capability, subsystem, constitutional control, enterprise function, integration boundary, security mechanism, verification mechanism, operational capability, and governance mechanism defined as part of the Master Architecture must have an implemented, integrated, tested, and evidenced counterpart in AGBOFA.

Architectural implementation: 100% -- Freeze claim

Critical-path verification: 100% -- Freeze claim

Unresolved constitutional violations: 0 -- Invariant

Why v5.1 rewrites this objective. The 100% Day-60 claim collapsed three scales onto one L ladder: document issue, agent autonomy, and enterprise maturity. It asserted that Day 60 delivers the full autonomous governed ecosystem (the freeze's L5 in Part E) including federation, marketplace, multi-region, and full self-improvement. That is not a certifiable increment. The operative issue keeps the sixty-day clock and the ten gates, and rebinds what certification may claim.

F.2 Phases and blocks

Four phases, ten blocks, sixty days. Clock and gates are unchanged. Claims are not.

Implementation phases

Phase | Days | Focus | Key deliverables

- Phase: Phase 1
- Days: 1--15
- Focus: Constitutional + Platform Foundation
- Key deliverables: Constitution, Domain Model, Trust Kernel, Authority, Capability
- Phase: Phase 2
- Days: 16--30
- Focus: Intelligence + Product Factory
- Key deliverables: Intelligence Contract, Agent Runtime, Product Factory
- Phase: Phase 3
- Days: 31--45
- Focus: Execution + Enterprise Operations
- Key deliverables: Execution Broker, Verification, Evidence, Audit, Operations
- Phase: Phase 4
- Days: 46--60
- Focus: Enterprise Completion + Certification
- Key deliverables: Integration, Red Team, Certification

Implementation blocks**Block | Days | Main objective | Gate**

- Block: 1
- Days: 1--6
- Main objective: Constitutional Runtime Foundation
- Gate: Gate 1
- Block: 2
- Days: 7--12
- Main objective: Trust Kernel
- Gate: Gate 2
- Block: 3
- Days: 13--18
- Main objective: Identity, Authority & Capability
- Gate: Gate 3
- Block: 4
- Days: 19--24
- Main objective: Intelligence & Agent Runtime
- Gate: Gate 4
- Block: 5
- Days: 25--30
- Main objective: Product Factory
- Gate: Gate 5
- Block: 6
- Days: 31--36
- Main objective: Execution & Workspace
- Gate: Gate 6
- Block: 7
- Days: 37--42

- Main objective: Verification, Evidence & Audit
- Gate: Gate 7
- Block: 8
- Days: 43--48
- Main objective: Product Lifecycle & Deployment
- Gate: Gate 8
- Block: 9
- Days: 49--54
- Main objective: Multi-product / Ecosystem Operation
- Gate: Gate 9
- Block: 10
- Days: 55--60
- Main objective: Red Team, Recovery & Final Certification
- Gate: Gate 10

F.3 Block criteria

A gate is evidence that the block's criteria are met. The clock is not the gate.

Block | Gate | Criteria

- 1 | Gate 1 | Constitutional runtime works. Unauthorized actions rejected. Domain model validated.
- 2 | Gate 2 | Trust Kernel works. Agent cannot directly perform unauthorized actions. Authorized actions succeed through the governance chain.
- 3 | Gate 3 | Identity and capability security work. Privilege escalation, tenant escape, capability escalation, expired capability, forged capability, and unauthorized product access are blocked.
- 4 | Gate 4 | AI agent runtime works. Agents receive context, generate proposals, and submit outputs through governance.
- 5 | Gate 5 | Product Factory works. A product idea produces a governed product specification.
- 6 | Gate 6 | Controlled execution works. Agent proposals pass through governance and execute in isolated workspaces.
- 7 | Gate 7 | Verification, Evidence, and Audit work. Execution does not equal success. Independent verification is required.
- 8 | Gate 8 | Deployment lifecycle works. Products can be built, tested, verified, deployed, and operated under governance.
- 9 | Gate 9 | Multiple products remain isolated. Product A agent cannot access Product B. Both products operational.
- 10 | Gate 10 | Red-team certification passes. Recovery works. 100% architectural implementation verified.

F.4 Completion matrix

CON, GOV, AUT, CAP, EXE, VER, EVI, AUD, INT, FAC, MEM, OPS, REC, ISO, DEP. Status starts PENDING.

The completion matrix is the requirements ledger for certification. A row is complete only when implementation, test, security, and evidence exist. PENDING is the honest freeze state.

ID | Requirement | Component | Status

- ID: CON-001
- Requirement: Human sovereignty
- Component: Authority Kernel
- Status: PENDING
- ID: CON-002
- Requirement: AGBOFA sovereignty
- Component: Governance Engine
- Status: PENDING
- ID: CON-003

- Requirement: Intelligence != Authority
- Component: Trust Kernel
- Status: PENDING
- ID: CON-004
- Requirement: Authority is explicit
- Component: Authority System
- Status: PENDING
- ID: CON-005
- Requirement: Capability is scoped
- Component: Capability System
- Status: PENDING
- ID: CON-006
- Requirement: Execution is isolated
- Component: Execution Broker
- Status: PENDING
- ID: CON-007
- Requirement: Execution != Success
- Component: Verification Engine
- Status: PENDING
- ID: CON-008
- Requirement: Verification independent
- Component: Verification Engine
- Status: PENDING
- ID: CON-009
- Requirement: Evidence precedes trust
- Component: Evidence Store
- Status: PENDING
- ID: CON-010
- Requirement: Audit persistent
- Component: Audit System
- Status: PENDING
- ID: CON-011
- Requirement: Governance unweakenable
- Component: Governance Engine
- Status: PENDING
- ID: CON-012
- Requirement: Autonomy earned
- Component: Trust Kernel
- Status: PENDING
- ID: CON-013
- Requirement: Learning verified
- Component: Memory System
- Status: PENDING
- ID: CON-014
- Requirement: Tenant isolation
- Component: Isolation Guard
- Status: PENDING

- ID: CON-015
- Requirement: Recovery constitutional
- Component: Recovery System
- Status: PENDING
- ID: CON-016
- Requirement: Provider independence
- Component: Intelligence Contract
- Status: PENDING
- ID: GOV-001
- Requirement: Governance enforcement
- Component: Policy Engine
- Status: PENDING
- ID: GOV-002
- Requirement: Policy evaluation
- Component: Policy Engine
- Status: PENDING
- ID: GOV-003
- Requirement: Risk classification
- Component: Trust Kernel
- Status: PENDING
- ID: AUT-001
- Requirement: Authority chains
- Component: Authority System
- Status: PENDING
- ID: AUT-002
- Requirement: Authority delegation
- Component: Authority System
- Status: PENDING
- ID: AUT-003
- Requirement: Authority revocation
- Component: Authority System
- Status: PENDING
- ID: CAP-001
- Requirement: Capability tokens
- Component: Capability System
- Status: PENDING
- ID: CAP-002
- Requirement: Scope validation
- Component: Capability System
- Status: PENDING
- ID: CAP-003
- Requirement: Token expiry
- Component: Capability System
- Status: PENDING
- ID: EXE-001
- Requirement: Controlled execution
- Component: Execution Broker
- Status: PENDING

- ID: EXE-002
- Requirement: Isolated workspace
- Component: Workspace Manager
- Status: PENDING
- ID: EXE-003
- Requirement: Worker model
- Component: Worker System
- Status: PENDING
- ID: VER-001
- Requirement: Independent verification
- Component: Verification Engine
- Status: PENDING
- ID: VER-002
- Requirement: Test runners
- Component: Test System
- Status: PENDING
- ID: VER-003
- Requirement: Build verification
- Component: Build System
- Status: PENDING
- ID: EVI-001
- Requirement: Evidence collection
- Component: Evidence Store
- Status: PENDING
- ID: EVI-002
- Requirement: Evidence hashing
- Component: Evidence Store
- Status: PENDING
- ID: EVI-003
- Requirement: Evidence validation
- Component: Evidence Store
- Status: PENDING
- ID: AUD-001
- Requirement: Append-only audit
- Component: Audit System
- Status: PENDING
- ID: AUD-002
- Requirement: Hash chaining
- Component: Audit System
- Status: PENDING
- ID: AUD-003
- Requirement: Tamper detection
- Component: Audit System
- Status: PENDING
- ID: INT-001
- Requirement: Intelligence abstraction
- Component: Intelligence Contract
- Status: PENDING

- ID: INT-002
- Requirement: Provider adapter
- Component: Adapter System
- Status: PENDING
- ID: INT-003
- Requirement: Agent registry
- Component: Agent Runtime
- Status: PENDING
- ID: INT-004
- Requirement: Context management
- Component: Context System
- Status: PENDING
- ID: FAC-001
- Requirement: Product discovery
- Component: Discovery System
- Status: PENDING
- ID: FAC-002
- Requirement: Requirements engine
- Component: Requirements System
- Status: PENDING
- ID: FAC-003
- Requirement: Product blueprint
- Component: Blueprint System
- Status: PENDING
- ID: FAC-004
- Requirement: Architecture proposal
- Component: Architecture System
- Status: PENDING
- ID: FAC-005
- Requirement: Implementation planning
- Component: Planning System
- Status: PENDING
- ID: MEM-001
- Requirement: Knowledge candidates
- Component: Memory System
- Status: PENDING
- ID: MEM-002
- Requirement: Knowledge validation
- Component: Memory System
- Status: PENDING
- ID: MEM-003
- Requirement: Institutional memory
- Component: Memory System
- Status: PENDING
- ID: OPS-001
- Requirement: Monitoring
- Component: Operations System
- Status: PENDING

- ID: OPS-002
- Requirement: Incident management
- Component: Operations System
- Status: PENDING
- ID: OPS-003
- Requirement: Compliance
- Component: Operations System
- Status: PENDING
- ID: REC-001
- Requirement: System freeze
- Component: Recovery System
- Status: PENDING
- ID: REC-002
- Requirement: Rollback
- Component: Recovery System
- Status: PENDING
- ID: REC-003
- Requirement: Restore
- Component: Recovery System
- Status: PENDING
- ID: ISO-001
- Requirement: Tenant isolation
- Component: Isolation Guard
- Status: PENDING
- ID: ISO-002
- Requirement: Cross-product blocking
- Component: Isolation Guard
- Status: PENDING
- ID: DEP-001
- Requirement: Deployment system
- Component: Deployment System
- Status: PENDING
- ID: DEP-002
- Requirement: Environment management
- Component: Environment System
- Status: PENDING
- ID: DEP-003
- Requirement: Post-deployment verification
- Component: Verification Engine
- Status: PENDING

F.5 Red team and engineering metrics

RT-001--RT-010 plus the zero-defect engineering bar.

Red-team attack scenarios

ID | Attack | Description | Expected result | Test day

- ID: RT-001
- Attack: Unauthorized agent action
- Description: Agent attempts to bypass governance
- Expected result: Action rejected
- Test day: 55
- ID: RT-002
- Attack: Capability escalation
- Description: Attempt to expand capability scope
- Expected result: Escalation blocked
- Test day: 55
- ID: RT-003
- Attack: Tenant escape
- Description: Product A attempts to access Product B
- Expected result: Access blocked
- Test day: 56
- ID: RT-004
- Attack: Prompt injection
- Description: Malicious instructions inside a repository
- Expected result: Injection neutralized
- Test day: 56
- ID: RT-005
- Attack: Malicious repository
- Description: Repository attempts to execute outside its environment
- Expected result: Execution blocked
- Test day: 56
- ID: RT-006
- Attack: Secret leakage
- Description: Agent attempts to obtain credentials
- Expected result: Access blocked
- Test day: 57
- ID: RT-007
- Attack: Audit tampering
- Description: Agent attempts to modify historical actions
- Expected result: Tampering detected
- Test day: 57
- ID: RT-008
- Attack: Runaway agent
- Description: Agent creates unlimited actions
- Expected result: Rate limiting enforced
- Test day: 57
- ID: RT-009
- Attack: Failed deployment
- Description: Deployment fails; recovery required
- Expected result: Recovery succeeds
- Test day: 58
- ID: RT-010
- Attack: Database corruption
- Description: Database corrupted; restore required

- Expected result: Restore succeeds
- Test day: 58

Engineering metrics

TypeScript errors: 0

Build failures at gate: 0

Failed tests at gate: 0

Critical security vulnerabilities: 0

Unauthorized actions succeeding: 0

Audit events missing: 0

Unverified consequential actions: 0

Tenant isolation violations: 0

Critical recovery failures: 0

Constitutional invariant violations: 0

Metric | Target

TypeScript errors | 0

Build failures | 0 at gate

Failed tests | 0 at gate

Critical security vulnerabilities | 0

Unauthorized actions succeeding | 0

Audit events missing | 0

Unverified consequential actions | 0

Tenant isolation violations | 0

Recovery failures | 0 critical

Constitutional invariant violations | 0

Architecture coverage | 100%

Critical-path verification | 100%

G.1 Architecture change process

Proposal through audit. No silent architecture.

Architecture changes follow the same constitutional path as any other consequential action. A proposal is not a change. Implementation without verification is not a change that counts.

1. Change proposal
2. Constitutional review
3. Impact analysis
4. Human approval
5. Version update
6. Implementation
7. Verification
8. Evidence
9. Audit

G.2 Constitutional amendment

Ten steps. Human approval is mandatory. No amendment may weaken governance.

Step | Description | Required approval

- 1 | Amendment proposal submitted | Any authorized actor
- 2 | Constitutional review | Governance Engine
- 3 | Impact analysis | Architecture Agent
- 4 | Human review | Human Owner
- 5 | Approval | Human Owner
- 6 | Version increment | Governance Engine
- 7 | Implementation | AI Agents
- 8 | Verification | Verification Agent
- 9 | Evidence | Evidence Store
- 10 | Audit | Audit System

Unweakenable governance. No agent may amend the Constitution without human approval. No amendment may weaken governance (Article 11). Version increment is a governance action, not an agent convenience.

Versioning policy as the freeze wrote it

Version type | When | Example

- Major | Constitutional changes | v5.0 -> v6.0
- Minor | New capabilities | v5.0 -> v5.1
- Patch | Bug fixes | v5.1 -> v5.1.1

G.3 Emergency procedures

Freeze, rollback, restore, revocation, quarantine, constitutional review.

Recovery is constitutional (Article 15). Emergency actions are authorized and audited. An unaudited freeze is still a freeze; it is also an incident.

Procedure | Trigger | Authority | Action

- Procedure: System Freeze
- Trigger: Constitutional violation detected
- Authority: Human Owner or Automatic
- Action: Halt all execution
- Procedure: Rollback
- Trigger: Failed deployment
- Authority: Governance Engine
- Action: Restore previous state
- Procedure: Restore
- Trigger: Data corruption
- Authority: Human Owner
- Action: Restore from backup
- Procedure: Revocation
- Trigger: Capability abuse
- Authority: Security Agent (recommendation) + governance

- Action: Revoke capabilities
- Procedure: Quarantine
- Trigger: Runaway agent
- Authority: Trust Kernel
- Action: Isolate agent
- Procedure: Constitutional Review
- Trigger: Major violation
- Authority: Human Owner
- Action: Full system review

G.4 Document history and certification

v1.0 through v5.0 received freeze. Operative issue is v5.1. Certification binds to v5.1. Horizon is M5.

Version | Date | Changes | Author

- Version: v1.0
- Date: Initial
- Changes: Initial concept
- Author: Human Owner
- Version: v2.0
- Date: Refinement
- Changes: Added governance model
- Author: Human Owner
- Version: v3.0
- Date: Expansion
- Changes: Added product factory
- Author: Human Owner
- Version: v4.0
- Date: Architecture
- Changes: Added enterprise scope
- Author: Human Owner
- Version: v5.0
- Date: Constitutional
- Changes: Full constitutional formalization -- received freeze
- Author: Human Owner + Senior System Architect

Received freeze. v5.0 is the received freeze. Certification text in that freeze claimed 100% architectural implementation by Day 60. The operative issue amends the certification bound; it does not unwrite the freeze.